NAME: BHUMI SINHA

SRN: PES2UG23CS127

SECTION: 6 – B

GEN AI HANDS-ON

[UNIT 2]

LangChain Foundations

LangChain is a framework used to build applications powered by Large Language Models (LLMs). It works similarly to Agno in the sense that we do not need to manually configure every setting for each LLM provider. Instead, we can use LangChain's built-in functions and simply specify the model we want to use. This makes it much easier to switch between different LLMs without rewriting the entire application logic.

Temperature

Temperature is a parameter that controls how random or creative the model's responses are. Its value usually ranges from **0 to 1**.

Observations from Temperature Tuning

When the same prompt was executed twice with the **temperature set to 1**, the model produced different responses each time because the randomness was high. The wording and sentence structure changed noticeably.

On the other hand, when the same prompt was run twice with the **temperature set to 0**, the output was almost identical both times. This happens because the model becomes more deterministic and generates the most likely next word consistently.

System Message

In the notebook example, a **System Message** was used to instruct the model to reply rudely and without proper grammar. This shows that system messages define the **behavior or style** of the model's response.

An LLM does not only process the user's prompt; it also takes into account instructions such as **system messages** and the **conversation history**.

In LangChain:

- A **System Message** gives instructions or sets the role/personality of the model.
- A **Human Message** is the actual input or question provided by the user.

So, the system message acts like a **persona or behavioral guide** for the LLM.

StrOutputParser

StrOutputParser is used to extract and return the final model response as plain text. It helps in cleaning and simplifying the output.

Pipeline

The basic flow of an LLM application can be understood as:

Input → Prompt → Model → Parser → Output

Instead of writing all these steps separately every time, LangChain allows us to connect them using the **pipe (|) operator**. This makes the workflow more organized and easier to use.

Prompt Engineering

Lazy Prompt vs Focused Prompt

A **lazy prompt** is a vague or incomplete prompt. For example, if we simply ask the model to "write a rejection mail," the model may assume its own reasons and details, even if they were never provided. This can lead to hallucinated or unnecessary information.

A **focused or structured prompt**, however, provides more context such as:

- who the email is addressed to,
- the reason for rejection,
- tone or style,
- any specific constraints.

Because of this extra clarity, the LLM is able to generate a more accurate, concise, and relevant response without making too many assumptions.

Zero-Shot, One-Shot, and Few-Shot Learning

In **Zero-Shot Learning**, we ask the model to solve a task **without giving any examples**. The model answers only based on what it has already learned during training.

In **One-Shot Learning**, we provide **one example** before asking the actual question.

In **Few-Shot Learning**, we provide **multiple examples** along with their expected outputs. These examples help the model understand the pattern and improve the quality of its responses.

Dynamic Few-Shot Prompting

Sometimes, we may have many examples, but not all of them can fit into the model's context window. In such cases, **dynamic few-shot prompting** is used.

Here, a **selector** chooses the most relevant examples from the available set and includes only those in the prompt. This makes prompting more efficient and ensures that the model gets only the most useful examples.

It also helps clearly separate the **instructions** from the **example data**, which improves prompt quality.

Advanced Prompting

1. Inner Monologue / Chain of Thought

I understood that LLMs often perform better when they are encouraged to solve problems **step by step** rather than giving a direct answer immediately. This intermediate reasoning helps reduce mistakes, especially in multi-step or logic-based problems.

2. Using Smaller Models

I learned that smaller models, such as **Llama 3.1 8B**, are useful for testing and experimentation. They make it easier to observe how prompt engineering techniques improve reasoning, since their mistakes are more visible.

3. Standard Prompt vs Chain of Thought

A normal prompt may sometimes lead to incomplete or incorrect answers. But if we use prompts like **"Think step by step"**, the model tends to break the problem into smaller parts, which often improves accuracy.

4. Latent Reasoning

I understood that LLMs work as **next-token predictors**, and reasoning emerges through the generation of intermediate tokens. These intermediate outputs provide context that helps the model arrive at a better final answer.

5. Tree of Thoughts (ToT)

Tree of Thoughts is a reasoning method where the model explores **multiple possible solution paths** before choosing the best one. It is similar to considering different options before making a final decision.

6. Graph of Thoughts (GoT)

Graph of Thoughts goes a step further than Tree of Thoughts by allowing the model to connect and combine multiple reasoning paths. Instead of following just one path, it can merge different ideas and generate a more refined answer.

RAG and Vector Stores

1. Retrieval-Augmented Generation (RAG)

I learned that LLMs do not always have complete or updated knowledge, and sometimes they may hallucinate. **RAG** improves reliability by retrieving relevant information from external documents and supplying it to the model before answer generation.

2. Embeddings and Vector Representation

Text cannot be directly understood by a computer in its raw form, so it is converted into **numerical vectors**, known as **embeddings**. These vectors represent the semantic meaning of the text.

3. Understanding Vector Space

Embeddings exist in a **multi-dimensional vector space**, where each dimension captures certain semantic characteristics of the text. This allows the system to compare meanings rather than just exact words.

4. Cosine Similarity

To measure how similar two pieces of text are, we use **cosine similarity**. It compares the angle between two vectors. If the angle is smaller, the meanings are considered more similar.

5. Naive RAG Pipeline

A simple RAG pipeline generally consists of three major steps:

1. **Retrieve** relevant documents
2. **Combine** them with the user query
3. **Generate** the final answer using the LLM

6. Knowledge Indexing using FAISS

FAISS is a vector database used to store embeddings efficiently and perform similarity search quickly. It helps retrieve the most relevant documents based on meaning rather than exact keyword matching.

7. Parametric vs Non-Parametric Memory

I understood that the knowledge already stored inside an LLM through training is called **parametric memory**. In contrast, the external information retrieved from documents is called **non-parametric memory**. Combining both helps generate more grounded and updated responses.

8. Indexing and Efficient Search

In large vector databases, comparing every stored vector with the query would be too slow. So, optimized indexing techniques are used to perform **fast nearest-neighbor search**, making retrieval much more efficient.